

Observabilidad integral sobre Google Kubernetes Engine: malla de servicios, detección de anomalías, señales de seguridad e ingeniería del caos

Fredy Orlando Pulido Quintero

Myriam Andrea Martínez Fontecha

Juan Francisco Javier Pérez Rivero

Nicolás Felipe Torres Amaya

Maestría en Arquitectura de Software

MASS – OBAP20264: Observabilidad en Ambientes Productivos

Maria Fernanda Ochoa Paipilla

31 de agosto de 2026

Observabilidad integral sobre Google Kubernetes Engine

Resumen ejecutivo

Se desplegó en Google Cloud un sistema de tres microservicios instrumentados con OpenTelemetry sobre Kubernetes con malla de servicios gestionada, con detección de anomalías por desviación estándar, cuatro señales de seguridad derivadas de registros de red y auditoría, y tres experimentos de ingeniería del caos. Toda la infraestructura se creó y se destruyó con Terraform.

El resultado más valioso del ejercicio no fue el tiempo de detección medido, sino los dos defectos que los experimentos revelaron en la propia instrumentación de medida. Un experimento que confirma lo esperado no informa sobre dónde está roto el instrumento; los dos que fallaron sí lo hicieron.

Arquitectura

La cadena de peticiones atraviesa tres procesos y una base de datos administrada: service-a recibe el checkout, llama por HTTP a service-b para reservar inventario, y este consulta a data-service, que lee el catálogo de productos en Cloud SQL. Tres saltos son el mínimo para que el trazado distribuido demuestre algo que no demostraría con dos: que el contexto se propaga más allá de un único salto entre procesos.

Cada servicio corre con dos contenedores en Kubernetes: la aplicación y un sidecar Envoy inyectado por Cloud Service Mesh. El OpenTelemetry Collector es la única excepción y está excluido de la malla de forma deliberada: si Envoy interceptara su tráfico, se generarían trazas sobre el envío de trazas.

La portabilidad del pipeline se sostiene en un hecho concreto y verificable: entre la configuración del Collector para el laboratorio local y la de Google Cloud solo cambian los

exportadores. Receptores, procesadores y tuberías son idénticos. La independencia de proveedor no es una afirmación de diseño, es un diff.

Tabla 1. Componentes desplegados

Componente	Tecnología	Función
service-a	FastAPI · Python 3.12	Checkout, expuesto por balanceador
service-b	FastAPI · Python 3.12	Reserva de inventario
data-service	FastAPI · Python 3.12	Catálogo, convenciones semánticas de BD
Collector	OTel Contrib 0.115	Recepción OTLP y exportación
Cloud SQL	PostgreSQL 15, IP privada	Inventario y catálogo
GKE	Estándar, 2 nodos e2-standard-2	Cómputo
Cloud Service Mesh	Gestionado, asm-managed	mTLS y telemetría de malla

Instrumentación y los tres pilares

La instrumentación combina el agente automático de OpenTelemetry con spans y métricas escritos a mano. El aporte específico de data-service son las convenciones semánticas de base de datos declaradas explícitamente, siguiendo la versión 1.28 de la especificación. No es un detalle formal: sin `db.collection.name` y `db.operation.name` todas las consultas caen en un mismo grupo y resulta imposible responder qué tabla se degradó.

La consulta se registra parametrizada, con el marcador y no con el valor. Si se registrara interpolada, cada ejecución sería un texto distinto —inútil para agrupar— y además se estarían escribiendo datos de cliente en la telemetría.

La correlación entre pilares quedó verificada de la forma más exigente posible: partiendo de un identificador de negocio. Se consulta Cloud Logging por el campo `cart_id` y el registro devuelve el identificador de traza en el campo nativo `trace` de la plataforma, lo que hace que la consola ofrezca un enlace directo a la traza distribuida. La correlación no es una convención propia que haya que explicar: la entiende la plataforma.

Tabla 2. Trazas de evidencia capturadas

Caso	Identificador de traza	Qué demuestra
Sana	89b5444cb33105a0...	Tres servicios en una traza; spans de negocio y de BD
Lenta (800 ms)	4c8c9f508d17face...	Un solo span concentra el tiempo; justifica el p99
Fallida (502)	94405bcd1bd2367f...	Span en ERROR con excepción dentro del span

El tercer caso merece una nota. La excepción se registra dentro del span y no fuera, por lo que su línea de registro lleva identificador de traza en vez de un valor nulo. Si el error se dejara escapar hasta el servidor de aplicación, el registro se emitiría ya fuera del contexto y quedaría huérfano, precisamente en el caso en que más falta hace poder correlacionarlo.

Detección de anomalías

La regla de alerta no usa un umbral fijo. Compara la tasa de error contra su propia media móvil de la última hora más dos desviaciones estándar, y exige además que el percentil 99 de latencia haya superado el objetivo de servicio. Un umbral fijo falla en dos direcciones opuestas a la vez: con tráfico alto, un uno por ciento son cientos de peticiones rotas y avisa tarde; con tráfico bajo, dos peticiones fallidas de veinte ya son un diez por ciento y avisa sin que ocurra nada.

La exigencia de dos señales simultáneas es lo que de verdad elimina ruido. Un rastreador pidiendo referencias inexistentes mueve la tasa de error sin que el servicio esté mal; una consulta pesada puntual mueve el percentil sin que nadie lo note. Que ambas se rompan a la vez distingue un incidente de una casualidad. La regla incorpora además una guarda de volumen mínimo, porque con muestras minúsculas cualquier valor parece anómalo: dos peticiones de madrugada con una fallida producen un cincuenta por ciento que supera cualquier sigma.

La misma expresión se ejecuta en los dos entornos. Al recibirse las métricas por Managed Service for Prometheus, la política en Google Cloud usa el mismo PromQL que el Prometheus local, de modo que la equivalencia entre entornos se puede comprobar en vez de suponerse.

Señales de seguridad

Los cuatro golden signals de la ingeniería de fiabilidad se trasladaron al dominio de seguridad: el tráfico son las conexiones entrantes externas, los errores son las conexiones que el cortafuegos rechaza, la saturación es el volumen de datos que sale hacia internet —la señal que delata una exfiltración— y la latencia es el tiempo hasta detectar un cambio de permisos. El valor de esta traducción es práctico: un equipo que ya lee paneles de fiabilidad puede leer este sin formación adicional, y la seguridad deja de vivir en una herramienta aparte.

Security Command Center no pudo activarse. Se activa a nivel de organización y el proyecto pertenece a una cuenta personal que no forma parte de ninguna, lo que se verificó por comando y no se supuso. Las cuatro señales lo sustituyen operando sobre registros de flujo, de cortafuegos y de auditoría, que sí funcionan a nivel de proyecto. La cobertura no es equivalente: la herramienta ausente correlaciona además con inteligencia de amenazas, y eso no tiene sustituto. Queda declarado como brecha.

Ingeniería del caos y tiempo de detección

Se ejecutaron tres experimentos. Los dos primeros no produjeron un tiempo de detección, cada uno por un motivo distinto, y ambos resultaron más informativos que el tercero.

Tabla 3. Experimentos de caos y resultados

Exp.	Inyección	Resultado medido	Hallazgo
1	200 ms de latencia en service-b (tc netem)	p99 de 247 a 4.900 ms · cero errores · sin alerta	La regla es ciega ante degradación pura de latencia
2	10 % de errores en data-	La herramienta reportó	El sidecar intercepta el

	service (proxy HTTP)	éxito · efecto nulo	tráfico antes que el proxy del caos
3	15 % fallos y 25 % latencia (aplicación)	15,1 % de error · p99 1.278 ms · alerta activada	La regla detecta lo que afirma detectar

El primer experimento multiplicó la latencia por veinte sin generar un solo error, y la alerta no se activó. El comportamiento es correcto —la regla exige las dos señales— pero mide por primera vez el costo de esa decisión de diseño. Exigir dos señales reduce el ruido y, a cambio, deja sin detectar una degradación severa de latencia. El dato convierte una preferencia de diseño en una compensación cuantificada.

El segundo experimento reprodujo un patrón conocido y peligroso: la herramienta de caos informó de una inyección exitosa que nunca tuvo efecto observable. Se detectó porque se buscaron los registros de degradación del servicio intermedio antes de interpretar el resultado. Si se hubiera aceptado el «cero errores» como éxito, se habría concluido que el sistema resiste cuando en realidad nada lo había puesto a prueba.

Tabla 4. Tiempo de detección, experimento 3

Métrica	Valor medido	Objetivo
Tasa de error alcanzada	15,1 %	—
Percentil 99 máximo	1.278 ms	—
Detección de la condición	3 s	—
Notificación al operador	≈ 63 s	< 120 s

El tiempo de detección se calcula sobre la serie temporal y no sobre la llegada del aviso por correo, que dependería del agrupamiento del gestor de alertas y de la latencia del proveedor, ninguno de los cuales es propiedad del sistema observado. Los tres segundos corresponden al instante en que la condición se vuelve cierta; la política exige que se mantenga sesenta segundos antes de notificar, para no alertar de picos transitorios, de modo que la cifra relevante para un operador es la segunda. Ambas son cotas inferiores.

Una limitación honesta del resultado: la línea base tenía errores prácticamente nulos, por lo que el umbral de dos sigmas degeneró en «cualquier error». El mecanismo estadístico está desplegado y es correcto, pero no quedó ejercitado. Demostrarlo exigiría una línea base ruidosa que este laboratorio no tiene.

Defectos encontrados por los experimentos

Dos fallos de la instrumentación de medida salieron a la luz, ambos capaces de haber invalidado las conclusiones sin dejar rastro.

El primero es una consulta que devuelve vacío en lugar de cero. Cuando no ha ocurrido ningún error, el selector de la métrica no coincide con ninguna serie y la suma devuelve un vector vacío; una división vacía deja la condición sin evaluar, ni verdadera ni falsa, sino inexistente. La alerta habría permanecido muda ante cualquier incidente que empezara desde un estado limpio, que es exactamente como empiezan casi todos.

El segundo es una comparación numérica errónea al leer el resultado: la condición devuelve el valor de la tasa de error y no un indicador binario, de modo que un truncamiento de decimales convertía un 0,009 verdadero en un cero falso. El tercer experimento llegó a reportarse como no detectado cuando la detección había funcionado en tres segundos.

Ninguno de los dos habría aparecido si los experimentos hubiesen salido bien a la primera. Es el argumento central a favor de la ingeniería del caos: su valor no está en confirmar que el sistema resiste, sino en descubrir que el instrumento con el que se mide no funcionaba.

Madurez de observabilidad

La autoevaluación sobre ocho dominios arroja una media de 3,25 sobre 5. La media importa menos que la forma del perfil: cinco de los ocho dominios están detenidos en el nivel 3 por el mismo motivo. Existen, están versionados y se reproducen, pero nadie les ha puesto un

número cuyo incumplimiento obligue a actuar. El problema no es de instrumentación, que está resuelta; medir es la parte fácil y decidir qué resulta inaceptable es la difícil.

Tabla 5. Autoevaluación de madurez

Dominio	Nivel	Qué lo frena
Tres pilares	4	La correlación se navega a mano
OpenTelemetry	4	Sin muestreo adaptativo
AIOps	3	La reducción de ruido no se ha medido
Observabilidad de red	3	Sin objetivo numérico sobre señales de red
DataOps	3	Sin atribución de costo por señal
Fiabilidad	3	El presupuesto de error no tiene consecuencia
Seguridad	2	Umbrales sin calibrar; herramienta no disponible
Resiliencia	4	Dominio más maduro: se rompió y se midió

La calificación más baja es la más honesta del informe. Los umbrales de las alertas de seguridad son cifras derivadas del criterio de quien escribió la infraestructura, no de observar semanas de comportamiento normal, y una alerta sin calibrar avisará de más o de menos sin que se sepa de cuál. La detección de anomalías se mantiene en nivel 3 y no en 4 aunque funcione, porque su tasa de falsos positivos frente al umbral estático todavía no se ha medido; mientras ese número no exista, la reducción de ruido es una hipótesis.

El plan a tres meses ataca primero lo que convierte hipótesis en números: medir falsos positivos durante treinta días, calibrar los umbrales de seguridad contra tráfico real e implementar el consumo del presupuesto de error con congelación de despliegues. El objetivo es una media de 4,1.

Control de costos

La infraestructura se creó con veinticuatro recursos de Terraform y se destruyó con una sola operación al terminar la jornada. El desmontaje elimina el balanceador antes que el clúster y

espera confirmación: en el orden inverso, la regla de reenvío queda huérfana fuera del estado de Terraform y sigue facturando sin aparecer en ningún inventario. Es la fuga de costo más común al desmontar Kubernetes administrado.

Conclusiones

El sistema demuestra los tres pilares emitidos, correlacionados y navegables desde un identificador de negocio, sobre una arquitectura de tres saltos con malla de servicios y base de datos administrada. La detección correlacionada funciona y su tiempo hasta notificar, sesenta y tres segundos, cumple el objetivo con holgura.

Con igual énfasis se documenta lo que no se logró: la herramienta de seguridad no pudo activarse por una restricción estructural de la cuenta, el mecanismo estadístico de la detección no quedó ejercitado por falta de una línea base ruidosa, y la regla resultó ciega ante una degradación de latencia sin errores. Los tres son resultados, no omisiones.

La lección transferible es metodológica. Los experimentos que fallaron aportaron más que el que funcionó, porque revelaron defectos en el instrumento de medida que ninguna revisión de código había detectado. Un sistema observable no es el que emite telemetría, sino aquel en el que se ha comprobado que la telemetría dice la verdad cuando algo se rompe.

Referencias

- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.
- Google Cloud. (2026). Cloud Service Mesh documentation. <https://cloud.google.com/service-mesh/docs>
- Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability engineering: Achieving production excellence. O'Reilly Media.
- OpenTelemetry Authors. (2026). Semantic conventions for database client calls (v1.28). <https://opentelemetry.io/docs/specs/semconv/database/>
- Rosenthal, C., & Jones, N. (2020). Chaos engineering: System resiliency in practice. O'Reilly Media.